









































































































Validation loss is computed with `torch.no_grad()` because **we are not training the model during validation, so we do not need to calculate or store gradients**. It saves memory and makes the evaluation faster. 🧠⚡

Let's break it down.

## During training

The model is learning, so we need gradients.

The process is:



Input  
↓  
Forward pass  
↓  
Prediction ( $\hat{y}$ )  
↓  
Calculate loss  
↓  
Backpropagation  
↓  
Calculate gradients  
↓  
Update weights

In code:

```
prediction = model(x)

loss = criterion(prediction, y)

loss.backward()      # calculate gradients
optimizer.step()     # update weights

# then
```

PyTorch needs to remember all the operations during the forward pass because it needs them later for:

$$\frac{dLoss}{dW}$$

## During validation

The model is **not learning**.

We only want:

"How well does my already-trained model perform on unseen data?"

The process is:

Input  
↓  
Forward pass

↓  
Prediction  
↓  
Calculate loss  
↓  
Done

There is no:

- `loss.backward()`
- `optimizer.step()`
- weight update

So gradients are unnecessary.

## What does `torch.no_grad()` do?

Normally, PyTorch builds a computation graph:

x  
↓  
Layer 1  
↓  
Layer 2  
↓  
 $\hat{y}$   
↓  
Loss

It remembers every operation:

```
"I multiplied here"  
"I added here"  
"I applied tanh here"
```

because it might need to go backwards.

When you use:

```
with torch.no_grad():  
    prediction = model(x)
```

Python

Run

you tell PyTorch:

```
"Only do the forward pass. Do not track information for backpropagation."
```

So the graph is not stored.

## Why does this matter?

### 1. Less memory usage

Without `torch.no_grad()` :

```
Forward pass:  
store activations  
store operations  
store gradients information
```

With `torch.no_grad()` :

```
Forward pass:  
make prediction  
throw away unnecessary training information
```

This is especially important for large neural networks.

### 2. Faster computation

Calculating and storing gradients takes time.

Validation only needs predictions, so skipping that work makes it faster.

## Example validation loop

```
model.eval()  
  
with torch.no_grad():  
    predictions = model(x_validation)
```

```
loss = criterion(predictions, y_validation)

print(loss.item())
```

Notice:

Python

Run

and:

Python

Run

do different things.

#### Purpose

`model.eval()`

Changes layer behaviour (e.g., dropout off, batch norm evaluation mode)

`torch.no_grad()`

Stops gradient calculation

You usually use both together.

## A common mistake

Some people think:

Python

Run

means gradients are disabled.

It does **not**.

This still calculates gradients:

```
model.eval()
```

```
prediction = model(x)  
loss = criterion(prediction, v)
```

Python

Run

`eval()` only changes the model's behaviour.

## Simple analogy

Training:

 "I am studying. I need to write notes so I can learn from my mistakes."

`loss.backward()` = writing notes about mistakes.

Validation:

 "I am taking a test. I only need my score."

`torch.no_grad()` = don't waste time writing study notes during the exam.

So the short answer:

Validation loss uses `torch.no_grad()` because validation only measures performance.

The model is not updating weights, so storing gradients is unnecessary and wastes memory and computation. 

     ...  Convert to PDF

what does the confusion matrix do

A **confusion matrix** is a table that shows how well a classification model's predictions match the actual answers. It helps you see what types of mistakes the model is making, not just whether it is right or wrong.  

A normal accuracy score only tells you:

"How many predictions were correct?"

A confusion matrix tells you:

"Which classes did the model confuse with each other?"

## Binary classification example

Imagine a model that predicts whether an email is:

- 0 = Not Spam
- 1 = Spam

After testing, we compare the model's predictions with the real labels.

The confusion matrix looks like:

|                  | Predicted: Not Spam | Predicted: Spam     |
|------------------|---------------------|---------------------|
| Actual: Not Spam | True Negative (TN)  | False Positive (FP) |
| Actual: Spam     | False Negative (FN) | True Positive (TP)  |

## What each part means

### 1. True Positive (TP)

The model correctly predicted the positive class.

Example:

Actual: Spam  
Prediction: Spam

✓ Correct

### 2. True Negative (TN)

The model correctly predicted the negative class.

Example:

Actual: Not Spam  
Prediction: Not Spam

✓ Correct

### 3. False Positive (FP)

The model predicted positive, but it was wrong.

Example:

Actual: Not Spam  
Prediction: Spam

✗ False alarm

This is also called a **Type I error**.

### 4. False Negative (FN)

The model predicted negative, but it was wrong.

Example:

Actual: Spam  
Prediction: Not Spam

✗ Missed detection

This is also called a **Type II error**.

## Example with numbers

Suppose your model tested 100 emails:

|                 | Predicted Not Spam | Predicted Spam |
|-----------------|--------------------|----------------|
| Actual Not Spam | 45                 | 5              |

|             | Predicted Not Spam | Predicted Spam |
|-------------|--------------------|----------------|
| Actual Spam | 10                 | 40             |

Meaning:

- 45 normal emails were correctly identified ✓
- 40 spam emails were correctly identified ✓
- 5 normal emails were incorrectly marked spam ✗
- 10 spam emails were missed ✗

## Why is this useful?

Because accuracy can hide problems.

Imagine a medical test:

1000 people:

- 990 are healthy
- 10 have a disease

A model predicts everyone is healthy:

Accuracy:

$$\frac{990}{1000} = 99\%$$

Looks amazing! 🎉

But the confusion matrix shows:

|                | Predicted Healthy | Predicted Disease |
|----------------|-------------------|-------------------|
| Actual Healthy | 990               | 0                 |
| Actual Disease | 10                | 0                 |

The model missed every sick person.

The accuracy was high, but the model is useless.



## Confusion matrix for multi-class problems

For more than two classes, you get a bigger table.

Example: Classifying animals:

|             | Cat | Dog | Bird |
|-------------|-----|-----|------|
| Actual Cat  | 40  | 3   | 2    |
| Actual Dog  | 5   | 35  | 0    |
| Actual Bird | 1   | 2   | 42   |

The diagonal shows correct predictions:

```
40
 35
   42
```

The off-diagonal values show mistakes.

For example:

```
Actual Cat → Predicted Dog: 3
```

means the model confused cats with dogs three times.

In PyTorch, you usually create one after validation:

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_true, y_pred)

# Print out confusion matrix
print(cm)
```

Python Run

For visualization:

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.heatmap(cm, annot=True, fmt="d")
plt.xlabel("Predicted")
plt.ylabel("Actual")
```

Python

Run

## For your neural network lab

If you are predicting continuous values like the **sine function**, you normally **do not use a confusion matrix** because that is a regression problem.

A confusion matrix is mainly for **classification**:

- Cat vs Dog 🐱 🐶
- Spam vs Not Spam 📧
- Disease vs No Disease 🩺
- Digit recognition (0-9) 📠

For your sine-wave feedforward network, the important plots are usually:

- ✓ Loss curve
- ✓ Predicted curve vs true sine curve
- ✓ Training/validation loss comparison

The confusion matrix would come in if your lab later changes to a classification dataset.

       **Convert to PDF**

so what differentiate neural networks from random forests

Great question. Both **neural networks** and **random forests** are machine learning models that learn patterns from data, but they learn in very different ways. 🌱 🧠 🌲

The biggest difference is:

A neural network learns a sequence of mathematical transformations using weights, while a random forest learns a collection of decision rules using many decision trees.

## 1. How they learn

### Neural Network 🧠

A neural network learns by adjusting **weights**.

Example:

$$\hat{y} = W_2 \tanh(W_1 x + b_1) + b_2$$

It starts with random weights:

```
graph TD
    Input --> Random_weights[Random weights]
    Random_weights --> Bad_prediction[Bad prediction]
    Bad_prediction --> Calculate_error[Calculate error]
    Calculate_error --> Adjust_weights[Adjust weights]
    Adjust_weights --> Better_prediction[Better prediction]
```

This happens through:

- Forward propagation
- Loss calculation
- Backpropagation
- Gradient descent

The model gradually learns a mathematical function that maps inputs → outputs.

### Random Forest 🌲

A random forest learns by building **many decision trees**.

A decision tree makes rules like:

```
Is age > 30?  
  |  
  Yes  
  |  
Is income > 50000?  
  |  
Prediction = Buy
```

A random forest creates many trees:

```
Tree 1 → prediction  
Tree 2 → prediction  
Tree 3 → prediction  
Tree 4 → prediction
```

↓

Final prediction (majority vote)

For classification:

Final answer = most common tree prediction

For regression:

Final answer = average of tree predictions

## 2. Structure difference

### Neural Network

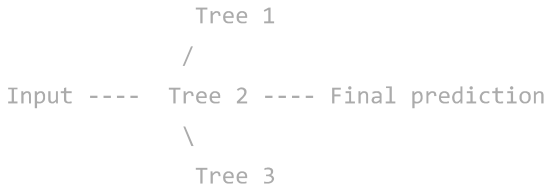
Looks like:

```
Input  
  |  
Hidden Layer  
  |  
Hidden Layer  
  |  
Output
```

Information flows through layers.

## Random Forest

Looks like:



Many independent trees vote together.

## 3. Type of patterns they learn

Neural networks are good at complex patterns

Especially when data is:

- Images 📷
- Audio 🎵
- Text 📄
- Large datasets

Example:

Recognizing a cat image:

The network learns:

Layer 1:  
Edges

Layer 2:  
Shapes

Layer 3:  
Eyes, ears

Layer 4:  
Cat features

Random forests are good at structured/tabular data

Examples:

| Age | Income | Credit Score | Loan |
|-----|--------|--------------|------|
| 25  | 30000  | 700          | No   |
| 45  | 90000  | 800          | Yes  |

A random forest learns rules like:

```
IF credit score > 750
```

```
AND income > 60000
```

```
THEN approve loan
```

## 4. Training difference

|                             | Neural Network   | Random Forest           |
|-----------------------------|------------------|-------------------------|
| Learning method             | Gradient descent | Decision tree splitting |
| Uses gradients?             | ✓ Yes            | ✗ No                    |
| Needs activation functions? | ✓ Yes            | ✗ No                    |
| Has weights?                | ✓ Yes            | ✗ No                    |
| Has trees?                  | ✗ No             | ✓ Yes                   |
| Training style              | Iterative        | Builds trees            |

## 5. Amount of data needed

### Neural Networks

Usually need a **lot of data**.

Example:

Image recognition:

10 images ❌

10,000 images ✅

They have many parameters to learn.

## Random Forest

Works well with smaller datasets.

Example:

1000 customer records

can still produce good results.

## 6. Interpretability

### Random Forest 🌲

More explainable.

You can inspect:

Income > 50000?

Age < 40?

Credit score high?

You can understand why it made a decision.

### Neural Network 🧠

Usually harder to interpret.

The network may have millions of weights:

Weight 1 = 0.238

Weight 2 = -0.091

Weight 3 = 0.771

...

It is often called a **black box**.

## 7. Example comparison

Suppose you want to predict house prices.

### Random Forest:

Learns:

IF house size > 2000 sq ft

AND location = Accra

THEN price is high

### Neural Network:

Learns:

Input features

↓

Layer transformations

↓

Complex mathematical relationship

↓

Price prediction

It may capture subtle interactions that are difficult to express as rules.

## Quick summary

Feature

Neural Network 

Random Forest 

Main idea

Learn mathematical  
representations

Combine many decision trees

Learning

Backpropagation

Tree splitting



| Feature                    | Neural Network 🧠               | Random Forest 🌲       |
|----------------------------|--------------------------------|-----------------------|
| Best for                   | Images, text, complex patterns | Tables, business data |
| Data needed                | Usually large                  | Small-medium          |
| Training                   | More tuning required           | Easier                |
| Explainability             | Low                            | Higher                |
| Handles nonlinear patterns | Yes                            | Yes                   |

A simple way to remember:

Random forest asks: "What rules separate these examples?" 🌲

Neural network asks: "What hidden mathematical pattern transforms these inputs into the answer?" 🧠

For your lab, you are studying neural networks because the focus is not just prediction, but how a model learns a function by adjusting internal parameters through gradients.

     ...  **Convert to PDF**

ChatGPT can make mistakes. Check important info.